Lab 10 Abstract Classes and Interfaces

Objective

In this lab, you will learn how to:

- Design and use abstract classes and abstract methods
- Specify common behavior for objects using interfaces
- Define interfaces and define classes that implement interfaces
- Define classes that implement interfaces using the implements keyword.
- Apply polymorphic referencing.
- Invoke methods through polymorphic referencing.

Current Lab Learning Outcomes (LLO)

By completion of the lab the students should be able to

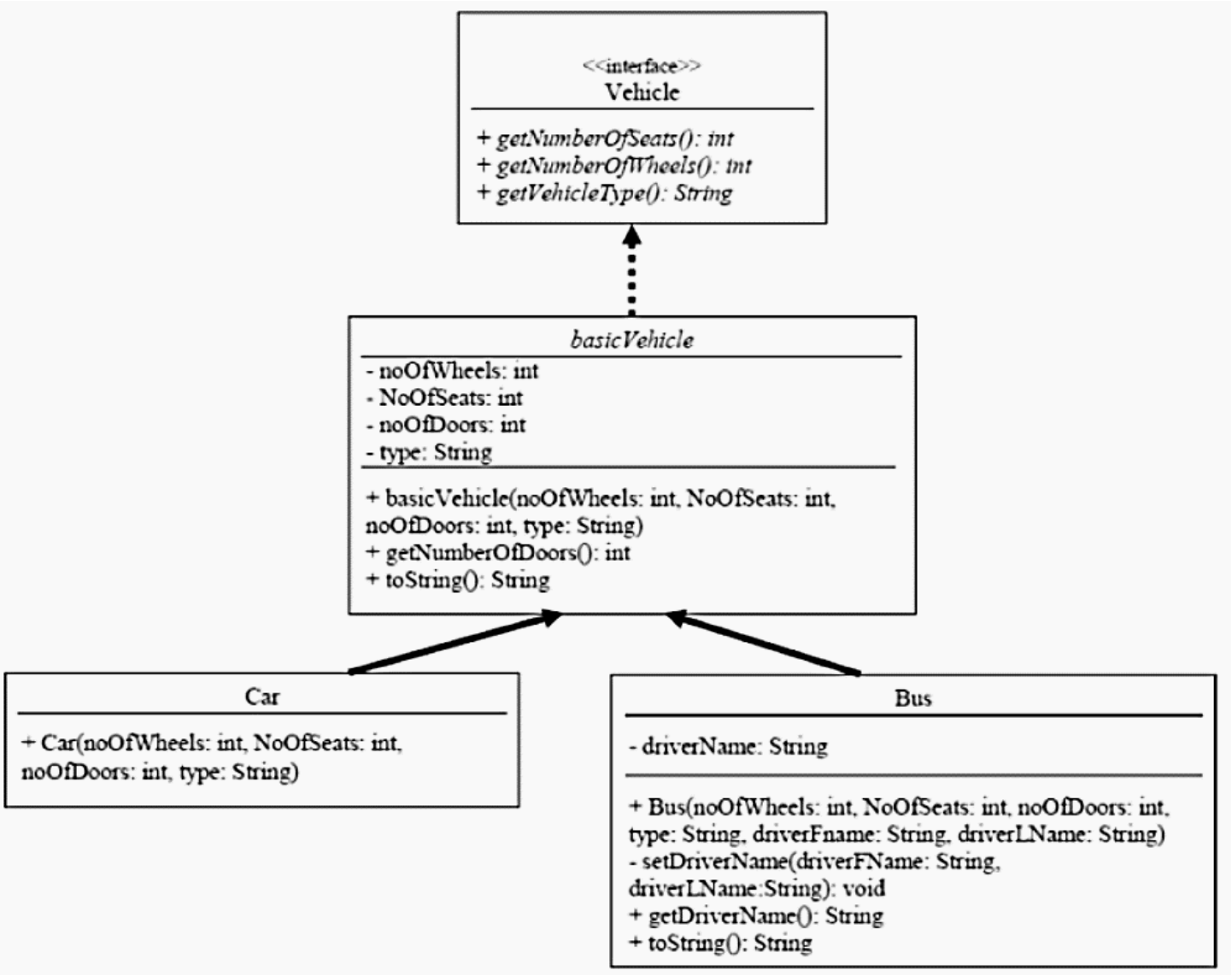
1. Apply interfaces and abstract classes to extend the functionality of classes
2. Interpret best practices in designing classes and class hierarchies from problem statements using sub classing, abstract classes, and interfaces to achieve polymorphism in object oriented software.

Lab Requirements

NetBeans IDE runs on the **Java** SE Development Kit (JDK) which consists of the **Java** Runtime Environment and developer tools for compiling, debugging, and running **applications.**

Activity:

Write a program that illustrates the UML diagrams and displays the output provided below.



The Sample Output:

```
Type: Car
 # Of Wheels : 4
 # of Seats : 5
 # of Doors : 2
 Type: Bus
 # Of Wheels : 35
 # of Seats : 6
 # of Doors : 4
 Driver :  BOB  Smith
```

Lab Description

The Vehicle Interface:

- The application has an interface named vehicle.
- The Vehicle interface has three abstract methods which are:

  1- getNumberOfSeats that returns the number of seats.

  2- getNumberOfWheels that returns the number of wheels.

  3- getVehicleType that returns the vehicle type.

The basicVehicle Abstract Class:

- The basicVehicle class **implements** the Vehicle interface.
- The basicVehicle class has 4 instance variables, which are: noOfWheels, noOfSeats, noOfDoors and type.
- The basicVehicle class has an argument constructor that initializes the instance variables.
- The basicVehicle class has a getter method that returns the numberOfDoors.
- The basicVehicle class overrides the toString method to create a string representation of its objects.

The Car Class:

- The Car class **extends** the basicVehicle class.
- The Car class has an argument constructor that initializes the instance variables through the invocation of the basicVehicle superclass constructor.

The Bus Class:

- The Bus class **extends** the basicVehicle class.
- The Bus class has an instance variable, which is: driverName.
- The Bus class has an argument constructor that initializes the instance variables through the invocation of the basicVehicle superclass constructor, in addition to initializing its instance variable driverName.
- The Bus class has a setter and getter method for the instance variable driverName.
- The Bus class overrides the toString method to create a string representation of its objects.

The Test Application Class:

Create an object of the class Car and the class Bus through polymorphic referencing and display their contents.